

Lab 3 Data Privacy Breach and Attacks

Objectives

In this lab, we will

1. Know basic use of Pandas.
2. Understand what are re-identification attack, reconstruction attack and differential attack.
3. Understand how to implement these attack methods.

1 Basic use of Pandas

1.1 What is Pandas?

pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, **real world** data analysis in Python. Additionally, it has the broader goal of becoming **the most powerful and flexible open source data analysis / manipulation tool available in any language**. It is already well on its way toward this goal.

1.2 Import Pandas

Importing a library means loading it into the memory and then it's there for you to work with. In order to import Pandas all you have to do is run the following code:

```
[1]: import pandas as pd # import the Pandas library
import numpy as np
```

1.3 Construct a dictionary

Example:

```
data = {'First Column Name':
        'Second Column Name': ['First value', 'Second value',...],
        ....
}
```

```
[2]: data = {'Name': ['Alice', 'Betty', 'Charles', 'David', 'Emily', 'Fred'],
            'Zipcode': [47677, 47602, 47678, 47905, 47909, 47906],
            'Age': [29, 22, 27, 43, 42, 47],
            'Sex': ['F', 'F', 'M', 'M', 'F', 'M'],
            'Disease': ['Ovarian Cancer', 'Ovarian Cancer',
                       'Prostate Cancer', 'Flu', 'Heart Disease', 'Heart Disease']}
```

1.4 Create a dataframe from dictionary

In order to convert a certain Python object (dictionary, lists etc) the basic command is:

```
pd.DataFrame()
```

Two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns).

```
[3]: df = pd.DataFrame(data)
df
```

```
[3]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease

1.5 View and inspect data

1. Get the first n rows

```
[4]: df.head(5)
```

```
[4]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease

2. Get the last n rows

```
[5]: df.tail(3)
```

```
[5]:
```

	Name	Zipcode	Age	Sex	Disease
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease

3. Get the number of rows and columns

```
[6]: df.shape
```

```
[6]: (6, 5)
```

4. Get the index, datatype and memory information.

```
[7]: df.info
```

```
[7]: <bound method DataFrame.info of      Name  Zipcode  Age Sex      Disease
0   Alice   47677   29  F   Ovarian Cancer
1   Betty   47602   22  F   Ovarian Cancer
2 Charles   47678   27  M  Prostate Cancer
3   David   47905   43  M           Flu
4   Emily   47909   42  F   Heart Disease
5    Fred   47906   47  M   Heart Disease>
```

5. Get the mean of all columns

```
[8]: df.mean(numeric_only=True)
```

```
[8]: Zipcode    47779.5
Age          35.0
dtype: float64
```

6. Get the correlation between columns in a data frame

```
[9]: df.corr(numeric_only=True)
```

```
[9]:      Zipcode      Age
Zipcode  1.000000  0.983538
Age       0.983538  1.000000
```

7. Get the highest value in each column

```
[10]: df.max()
```

```
[10]: Name          Fred
Zipcode         47909
Age             47
Sex             M
Disease  Prostate Cancer
dtype: object
```

1.6 Selection of Data

1. Select columns

```
[11]: df['Sex']
```

```
[11]: 0    F
      1    F
      2    M
      3    M
      4    F
      5    M
      Name: Sex, dtype: object
```

```
[12]: df[['Age', 'Sex']]
```

```
[12]:   Age Sex
      0  29  F
      1  22  F
      2  27  M
      3  43  M
      4  42  F
      5  47  M
```

2. Select rows by position

```
[13]: df.iloc[0]
```

```
[13]: Name          Alice
      Zipcode      47677
      Age         29
      Sex          F
      Disease   Ovarian Cancer
      Name: 0, dtype: object
```

```
[14]: df.iloc[:3]
```

```
[14]:   Name  Zipcode  Age Sex      Disease
      0  Alice   47677   29  F   Ovarian Cancer
      1  Betty   47602   22  F   Ovarian Cancer
      2 Charles   47678   27  M   Prostate Cancer
```

3. Select an element

```
[15]: df.iloc[0,0]
```

```
[15]: 'Alice'
```

4. Select rows by index

```
[16]: df.loc[0]
```

```
[16]: Name          Alice
      Zipcode      47677
```

```
Age                29
Sex                F
Disease    Ovarian Cancer
Name: 0, dtype: object
```

```
[17]: df.loc[:3]
```

```
[17]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu

```
[18]: df.loc[0, 'Name']
```

```
[18]: 'Alice'
```

5. Replace values with different values

Replace values given in `to_replace` with `value`. Values of the DataFrame are replaced with other values dynamically.

```
df.replace(to_replace=None, value=None, inplace=False)
```

```
[19]: df.replace('Flu', 'Lung Cancer')
```

```
[19]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Lung Cancer
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease

2 Implementation of Re-identification / De-anonymisation

2.1 Linkage Attack

We now create a small dataset and see how to implement a linkage attack. We follow the mini dataset from the former introduction.

```
[20]: data = {'Name': ['Alice', 'Betty', 'Charles', 'David', 'Emily', 'Fred',  
                    'Tom', 'Alex', 'Vincent', 'Serena'],  
            'Zipcode': [47677, 47602, 47678, 47905, 47909, 47906,  
                        47433, 47489, 47422, 47443],  
            'Age': [29, 22, 27, 43, 42, 47,  
                   32, 33, 34, 38],  
            'Sex': ['F', 'F', 'M', 'M', 'F', 'M',  
                   'M', 'M', 'M', 'F']
```

```
'Disease': ['Ovarian Cancer', 'Ovarian Cancer',
'Prostate Cancer', 'Flu', 'Heart Disease', 'Heart Disease',
'Flu', 'Flu', 'Flu', 'Heart Disease']}]}
```

```
[21]: df = pd.DataFrame(data)
df
```

```
[21]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease
6	Tom	47433	32	M	Flu
7	Alex	47489	33	M	Flu
8	Vincent	47422	34	M	Flu
9	Serena	47443	38	F	Heart Disease

Firstly, we remove its key attribute “name”,

```
[22]: data1 = {'Zipcode': [47677, 47602, 47678, 47905, 47909, 47906,
47433,47489,47422,47443],
'Age': [29, 22, 27, 43, 42, 47,
32,33,34,38],
'Sex': ['F', 'F', 'M', 'M', 'F', 'M',
'M', 'M', 'M', 'F'],
'Disease': ['Ovarian Cancer', 'Ovarian Cancer',
'Prostate Cancer', 'Flu', 'Heart Disease', 'Heart Disease',
'Flu', 'Flu', 'Flu', 'Heart Disease']}]}
```

```
[23]: df1 = pd.DataFrame(data1)
df1
```

```
[23]:
```

	Zipcode	Age	Sex	Disease
0	47677	29	F	Ovarian Cancer
1	47602	22	F	Ovarian Cancer
2	47678	27	M	Prostate Cancer
3	47905	43	M	Flu
4	47909	42	F	Heart Disease
5	47906	47	M	Heart Disease
6	47433	32	M	Flu
7	47489	33	M	Flu
8	47422	34	M	Flu
9	47443	38	F	Heart Disease

Secondly, we remove its sensitive attribute “Disease” as the second dataset

```
[24]: data2 = {'Name': ['Alice', 'Betty', 'Charles', 'David', 'Emily', 'Fred',
'Tom', 'Alex', 'Vincent', 'Serena'],
'Zipcode': [47677, 47602, 47678, 47905, 47909, 47906,
47433,47489,47422,47443],
'Age': [29, 22, 27, 43, 42, 47,
32,33,34,38],
'Sex': ['F', 'F', 'M', 'M', 'F', 'M',
'M', 'M', 'M', 'F']}
```

```
[25]: df2 = pd.DataFrame(data2)
df2
```

```
[25]:
```

	Name	Zipcode	Age	Sex
0	Alice	47677	29	F
1	Betty	47602	22	F
2	Charles	47678	27	M
3	David	47905	43	M
4	Emily	47909	42	F
5	Fred	47906	47	M
6	Tom	47433	32	M
7	Alex	47489	33	M
8	Vincent	47422	34	M
9	Serena	47443	38	F

So when an adversary has both datasets, the adversary may re-identify that “Alice” and “Betty” have “Ovarian Cancer”, “Charles” has “Prostate Cancer”, and so on.

2.2 Unsorted Matching Attack

Now, let’s create three datasets to see how to implement unsorted matching attack.

```
[26]: data_unsorted1 = {"Sex": ["Female", "Female", "Female", "Female", "Female",
↪ "Female", "Male", "Male", "Male", "Male", "Male"],
"ZIP": ['02138', '02139', '02141', '02142', '02138', '02139',
↪ '02141', '02142', '02138', '02139', '02141']}
```

```
[27]: df3 = pd.DataFrame(data_unsorted1)
df3
```

```
[27]:
```

	Sex	ZIP
0	Female	02138
1	Female	02139
2	Female	02141
3	Female	02142
4	Female	02138
5	Female	02139
6	Male	02141
7	Male	02142

```

8      Male  02138
9      Male  02139
10     Male  02141

```

Now, let's build the second dataset of these three, We anonymise its first attribute "Sex".

```

[28]: data_unsorted2 = {"Sex" : ["Person", "Person", "Person", "Person", "Person", "Person", "Person", "Person", "Person", "Person"],
    ↪ "ZIP" : ['02138', '02139', '02141', '02142', '02138', '02139', '02141', '02141', '02138', '02139']}

```

```

[29]: df4 = pd.DataFrame(data_unsorted2)
df4

```

```

[29]:      Sex  ZIP
0  Person 02138
1  Person 02139
2  Person 02141
3  Person 02142
4  Person 02138
5  Person 02139
6  Person 02141
7  Person 02141
8  Person 02138
9  Person 02139

```

Now, let's build the third dataset of these three. We anonymise its second attribute "ZIP".

```

[30]: data_unsorted3 = {"Sex" : ["Female", "Female", "Female", "Female", "Female", "Female", "Male", "Male", "Male", "Male"],
    ↪ "ZIP" : ['02130', '02130', '02140', '02140', '02130', '02130', '02140', '02140', '02130', '02140']}

```

```

[31]: df5 = pd.DataFrame(data_unsorted3)
df5

```

```

[31]:      Sex  ZIP
0  Female 02130
1  Female 02130
2  Female 02140
3  Female 02140
4  Female 02130
5  Female 02130
6   Male  02140
7   Male  02140
8   Male  02130
9   Male  02130

```



```
10    Male  02140
11    Male  02140
```

```
[32]: data_unsorted4 = {"Sex1" : ["Female", "Female", "Female", "Female", "Female",
    ↪ "Female", "Male" , "Male" , "Male" , "Male" , "Male" , "Male" , "Male"],
    "ZIP1" : ["02138" , "02139" , "02141" , "02142" , "02138" ,
    ↪ "02139" , "02141" , "02142" , "02138" , "02139" , "02141" , "02142"],
    "Sex2" : ["Person", "Person", "Person", "Person",
    ↪ "Person", "Person" , "Person", "Person", "Person", "Person", "Person",
    ↪ "Person"],
    "ZIP2" : ["02130" , "02130" , "02140" , "02140" , "02130" ,
    ↪ "02130" , "02140" , "02140" , "02130" , "02130" , "02140" , "02140"],
    "Sex3" : ["Female", "Female", "Female", "Female", "Female",
    ↪ "Female", "Male" , "Male" , "Male" , "Male" , "Male" , "Male"],
    "ZIP3" : ["02138" , "02139" , "02141" , "02142" , "02138" ,
    ↪ "02139" , "02141" , "02142" , "02138" , "02139" , "02141" , "02142"]}
```

```
[33]: df6 = pd.DataFrame(data_unsorted4)
df6
```

```
[33]:
```

	Sex1	ZIP1	Sex2	ZIP2	Sex3	ZIP3
0	Female	02138	Person	02130	Female	02138
1	Female	02139	Person	02130	Female	02139
2	Female	02141	Person	02140	Female	02141
3	Female	02142	Person	02140	Female	02142
4	Female	02138	Person	02130	Female	02138
5	Female	02139	Person	02130	Female	02139
6	Male	02141	Person	02140	Male	02141
7	Male	02142	Person	02140	Male	02142
8	Male	02138	Person	02130	Male	02138
9	Male	02139	Person	02130	Male	02139
10	Male	02141	Person	02140	Male	02141
11	Male	02142	Person	02140	Male	02142

3 Differential Attack

Now we will implement differential attack using the example introduced in the lecture. Firstly, we create a raw dataset without any anonymity.

```
[34]: data_da = {"Name" : ["Alice", "Bob", "Chris", "David", "Eric", "Fiona",
    ↪ "Grace"],
    "Grade" : [85 , 62 , 70, 93 , 58, 81 , 74]}
```

```
[35]: df8 = pd.DataFrame(data_da)
df8
```

```
[35]:
```

	Name	Grade
0	Alice	85
1	Bob	62
2	Chris	70
3	David	93
4	Eric	58
5	Fiona	81
6	Grace	74

When we consider the attribute 'Grade' is sensitive, the dataset is anonymised as following:

```
[36]: data_da1 = {"Name" : ["Alice", "Bob", "Chris", "David", "Eric", "Fiona", "Grace"],
                  "Grade" : ["**", "**", "**", "**", "**", "**", "**"]}
```

```
[37]: df9 = pd.DataFrame(data_da1)
df9
```

```
[37]:
```

	Name	Grade
0	Alice	**
1	Bob	**
2	Chris	**
3	David	**
4	Eric	**
5	Fiona	**
6	Grace	**

However, we can still query about the raw dataset, for example, we want to know the sum of all students' grade, it will return the result to us.

```
[38]: df8['Grade'].sum()
```

```
[38]: 523
```

We know that the sum of all students' grade is 523. Now, we will query about the sum of the first 6 students' grade except Grace, it will return the result to us.

```
[39]: df8['Grade'].sum() - df8['Grade'][6]
```

```
[39]: 449
```

Now, we can use the first query answer to subtract the second query answer to get Grace's grade

```
[40]: 523 - 449
```

```
[40]: 74
```

It returns Grace's grade is 74.

4 Conclusion

In this lab, we learnt:

1. Pandas and DataFrame;
2. How to build a dataframe and show the dataset;
3. Re-identification attack, re-construction attacks and differential attack.